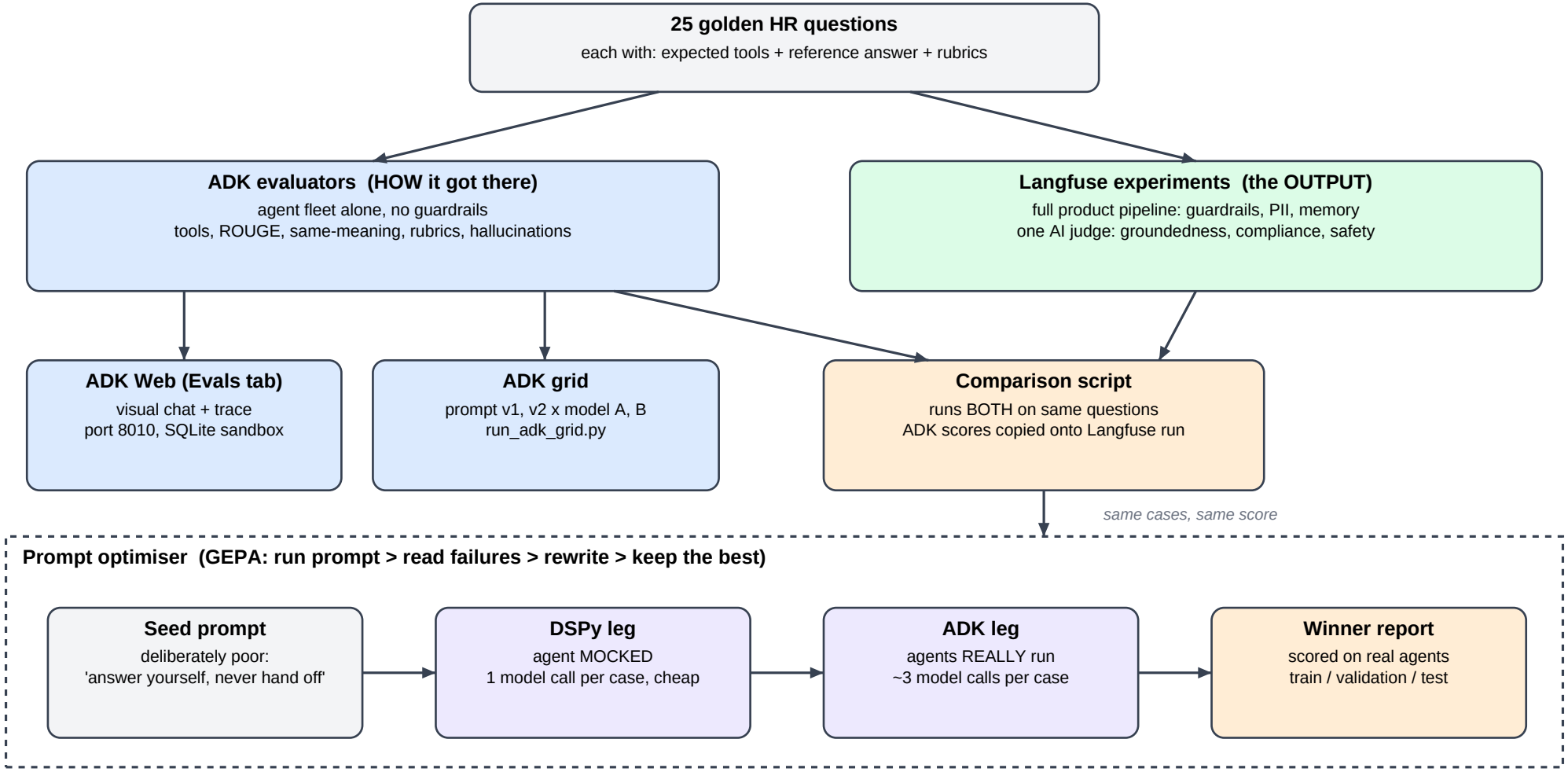
How our evaluation and prompt-optimisation setup fits together

Same 25 questions, two ways of judging them, and a prompt optimiser that reuses both.



1. Google ADK evaluation (ADK Web, CLI, comparison)

File	How it relates	See it manually
`backend/app/evaluations/adk_expectations.py`	The test material in ADK format: expected tools per case, rubrics, and the ADK criteria config (which built-in evaluators run, thresholds, judge model).	Open it; or run **python backend/scripts/build_adk_evalset.py** to regenerate the eval files.
`backend/app/evaluations/adk_support.py`	Runs an eval set through ADK's own evaluation service on the same HR agent; returns every metric per case. Also the 'identity shim' that gives the agent the employee header.	Used by every ADK script below.
`backend/adk_web/peopleops/agent.py`	The entry point ADK Web loads. Re-exports the SAME root agent, forces an isolated SQLite database, applies the identity shim.	Started via **npm run adk:web**.
`backend/adk_web/peopleops/*.evalset.json, eval_config.json`	Generated eval sets (10 deterministic, 25 golden) and the criteria file. The Evals tab lists these.	ADK Web: pick peopleops, then Evals tab.
`backend/scripts/run_adk_web.py`	Starts the ADK dev UI on the HR agents.	**npm run adk:web**, open http://localhost:8010/dev-ui
`backend/scripts/run_adk_eval.py`	Command-line version of the Evals tab: prints every score and each rubric per case.	**python backend/scripts/run_adk_eval.py** (3 cases; --all for everything)
`backend/scripts/build_adk_evalset.py`	Regenerates the eval-set and criteria JSON from the golden cases.	Run it after changing cases or rubrics.
`backend/app/evaluations/adk_comparison.py`	Runs ADK evaluators and the Langfuse pipeline on the same cases, builds the side-by-side report, copies ADK scores onto the Langfuse run.	Used by the script below and the API endpoint.
`backend/scripts/run_adk_comparison.py`	Prints the side-by-side table and a Langfuse run link.	**python backend/scripts/run_adk_comparison.py**
`backend/app/core/llm_bridge.py`	Chooses and builds the model for the agents (Groq / OpenAI / mock). Holds the Groq fix for ADK 2.x and the key export used by ADK judges.	Read-only for you; no screen.
`backend/app/core/employee_context.py`	Builds the '[Active Employee ...]' header and current-time line the real pipeline gives every message.	No screen.
`backend/.venv-adk/ , backend/requirements-eval.txt, constraints-windows.txt`	The ADK 2.13 install (separate folder) and the package lists that produced it.	**npm run adk:web** uses this folder first.

2. Langfuse evaluation (experiments, judge, webhook)

File	How it relates	See it manually
`backend/app/evaluations/llm_judge_service.py`	The AI judge that scores each answer (groundedness, helpfulness, compliance, safety) plus the keyword fallback.	Scores appear in Langfuse runs.
`backend/app/evaluations/golden_dataset_25.py`	The 25 golden questions with expected keywords, forbidden terms and statutory clauses. The shared test material.	Open the file.
`backend/app/services/dataset_sync_service.py`	Syncs the golden cases to the Langfuse dataset and runs the golden benchmark as an experiment run.	Langfuse: Datasets, then peopleops-golden-benchmark-25, then Runs.
`backend/app/evaluations/grid_config.py, grid_evaluator.py`	Langfuse prompt-version x model grid: parses the config and runs each cell.	API: POST /api/v1/agents/experiments/grid
`backend/app/services/webhook_auth.py`	Verifies the signed webhook Langfuse sends when you click 'Run experiment' (through ngrok).	Langfuse: dataset, Custom Experiment trigger.

File	How it relates	See it manually
`backend/app/services/langfuse_prompt_service.py`	Prompt versions and labels stored in Langfuse; the agents read them.	Langfuse: Prompts.
`backend/app/observability/langfuse_client.py`	The one shared Langfuse connection (masking, environment, release, flush).	Traces appear in Langfuse: Tracing.
`backend/app/evaluations/eval_isolation.py`	Keeps evaluation runs from polluting the real database.	No screen.
`backend/app/api/v1/agents.py`	API endpoints: experiments/grid, experiments/adk-compare (+ /latest), observability/sync-all, feedback.	http://localhost:38290/docs (Swagger) when the backend runs.

3. Prompt optimisation and the version x model grid

File	How it relates	See it manually
`backend/app/optimization/common.py`	What both legs share: the cases, the train/validation/test split, the seed prompts, the scoring.	Open it.
`backend/app/optimization/metric.py`	The one score per case (half 'right steps', half 'good answer') and the failure notes the reflection model reads.	Open it.
`backend/app/optimization/dspy_leg.py`	DSPy leg: runs dspy.GEPA against a MOCKED agent (one hand-off decision, one model call per case).	Part of **npm run optimize:prompt**.
`backend/app/optimization/adk_leg.py`	ADK leg: runs ADK's GEPA prompt optimizer on the REAL agents.	Part of **npm run optimize:prompt**.
`backend/app/optimization/pipeline.py`	Runs DSPy, then ADK, scores the seed and both winners on the real agents, writes the report.	Report: backend/reports/optimization_*.md
`backend/app/optimization/runtime.py`	Sets up the isolated ADK runtime before the optimisers start.	No screen.
`backend/app/optimization/adk_grid.py`	The prompt-version x model grid on ADK evaluators (ADK cannot do this natively).	Report: backend/reports/adk_grid_*.json
`backend/scripts/run_prompt_optimization.py`	Command to start the optimisation.	**npm run optimize:prompt** (options: --mode dspy|adk|both, --dspy-calls, --adk-calls, --seed)
`backend/scripts/run_adk_grid.py`	Command to start the grid.	**npm run adk:grid -- --versions v12,v13 --models openai/gpt-4o-mini,groq/openai/gpt-oss-120b**
`backend/app/evaluations/prompt_optimizer.py`	Older Langfuse-side prompt optimiser (used by the Langfuse grid); guards the live label.	Used by the Langfuse grid option.
`backend/requirements-optimize.txt`	Packages for the optimiser (dspy, gepa; same GEPA version in both legs).	Install before the first run.
`docs/prompt_optimization.md`, `docs/adk_web_and_langfuse.md`, `docs/langfuse_integration.md`	Written explanations of each part.	Open in any editor.
`backend/reports/`	Results of past runs (optimisation report, grid reports).	Open optimization_20261026_195446.md first.
`backend/tests/test_adk_web_and_comparison.py`, `test_prompt_optimization.py`, `test_langfuse_*.py, test_webhook_*.py`	Automated tests for these parts.	**pytest backend/tests**

Quick ways to look at it yourself

ADK screen: npm run adk:web, then http://localhost:8010/dev-ui, pick peopleops, Evals tab. **Langfuse screen:** Datasets, peopleops-golden-benchmark-25, Runs. **Side by side:** python backend/scripts/run_adk_comparison.py (prints a Langfuse run link). **Optimiser result:** open backend/reports/optimization_20261026_195446.md. **API docs:** http://localhost:38290/docs

while the backend runs.